ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«Самарский национальный исследовательский университет имени академика С.П. Королева»

ОТЧЁТ

по лабораторной работе №2

Нормы матриц, обусловленность, решение СЛАУ методом Гаусса

Исполнитель:
Барабанов Иван Сергеевич
Вариант №16

Группа: 6312-100503D
Преподаватель: Подклетнова С.В.

Самара, 2026

Цель работы

Изучить понятия норм векторов и матриц, научиться вычислять различные нормы матрицы, исследовать число обусловленности и его влияние на точность решения СЛАУ, реализовать метод Гаусса с выбором главного элемента, а также исследовать рост числа обусловленности матрицы Гильберта с увеличением её размера.

Теоретические сведения

Нормы векторов и матриц

Норма вектора $\|\mathbf{x}\|$ — это обобщение понятия длины, удовлетворяющее трём аксиомам: неотрицательность ($\|\mathbf{x}\| \geq 0$, причём $\|\mathbf{x}\| = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$), однородность ($\|\alpha\mathbf{x}\| = |\alpha| \cdot \|\mathbf{x}\|$) и неравенство треугольника ($\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$).

Основные нормы вектора:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|, \quad \|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}, \quad \|\mathbf{x}\|_\infty = \max_i |x_i|$$

Норма матрицы, согласованная с нормой вектора, определяется как

$$\|A\| = \max_{\mathbf{x} \neq 0} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|}$$

Часто используемые матричные нормы:

- 1-норма (столбцовая): $\|A\|_1 = \max_j \sum_i |a_{ij}|$
- ∞-норма (строковая): $\|A\|_\infty = \max_i \sum_j |a_{ij}|$
- Норма Фробениуса: $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$

Число обусловленности матрицы

Число обусловленности матрицы A относительно нормы $\|\cdot\|$:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|$$

Оно характеризует чувствительность решения СЛАУ $Ax = b$ к возмущениям входных данных. Если $\kappa(A)$ велико, малые ошибки в b или A приводят к большим ошибкам в x. Матрица считается плохо обусловленной при $\kappa \gg 1$.

Метод Гаусса с выбором главного элемента

Метод Гаусса состоит из прямого хода (приведение матрицы к верхнетреугольному виду) и обратного хода (последовательное нахождение неизвестных). Выбор главного элемента по столбцу заключается в поиске максимального по модулю элемента в текущем столбце среди строк с i-й по n-ю, перестановке соответствующей строки на диагональ и последующем исключении. Это предотвращает деление на малые числа и повышает устойчивость алгоритма к ошибкам округления.

Матрица Гильберта

Матрица Гильберта $H_{ij} = \frac{1}{i+j-1}$ — классический пример плохо обусловленной матрицы. Её столбцы становятся почти линейно зависимыми с ростом n, а число обусловленности растёт экспоненциально: $\kappa(H_{10}) \sim 10^{13}$.

Листинг программы

Листинг 1: Исходный код программы (Python)

```
#
#                  16

import numpy as np
import matplotlib.pyplot as plt

print("=" * 60)
print("                              ␣              ␣  2  ")
print("=" * 60)

# ========================================
#                   1  :
# ========================================

A = np.array([
    [3.0, 1.0, 1.0, 1.0, 1.0],
    [1.0, 3.0, 1.0, 1.0, 1.0],
    [1.0, 1.0, 3.0, 1.0, 1.0],
    [1.0, 1.0, 1.0, 3.0, 1.0],
    [1.0, 1.0, 1.0, 1.0, 3.0],
])

print("\n---␣                ␣  1  :␣            ␣                 ␣---")
print("                ␣   :")
print(A)

#                                                    numpy
norm1 = np.linalg.norm(A, ord=1)
norm_inf = np.linalg.norm(A, ord=np.inf)
norm_fro = np.linalg.norm(A, ord='fro')

print(f"1-           :␣␣␣␣␣␣␣␣␣␣␣␣␣␣{norm1:.4f}")
print(f"                        ␣            :␣␣␣␣{norm_inf:.4f}")
print(f"           ␣                      :␣␣␣␣␣{norm_fro:.4f}")

# ========================================
#                   2  :
# ========================================

cond_1 = np.linalg.cond(A, p=1)
cond_inf = np.linalg.cond(A, p=np.inf)
cond_fro = np.linalg.cond(A, p='fro')

print("\n---␣               ␣2:␣           ␣
                                  ␣---")
print(f"cond_1␣(     ␣1-           ):␣{cond_1:.4f}")
print(f"cond_inf␣(     ␣                      ␣           ):␣{cond_inf:.4
    f}")
print(f"cond_fro␣(     ␣           ␣                    ):␣{cond_fro:.4f}
    ")
```

```
# ========================================
#                  3  :

# ========================================

def gauss_solve(A, b):
    '''
␣␣␣␣            ␣        ␣Ax␣=␣b␣              ␣            ␣  ␣

␣␣␣␣                ␣                ␣    ␣              ␣(
                     ␣          ).
␣␣␣␣                    ␣            ␣              ␣x.
␣␣␣␣'''
    n = len(A)
    a = A.astype(float).copy()
    bb = b.astype(float).copy()

    #
    for i in range(n):
        #
                        i
        max_row = i
        max_val = abs(a[i][i])
        for k in range(i + 1, n):
            if abs(a[k][i]) > max_val:
                max_val = abs(a[k][i])
                max_row = k

        #
        if max_row != i:
            a[[i, max_row]] = a[[max_row, i]]
            bb[[i, max_row]] = bb[[max_row, i]]

        #
        if abs(a[i][i]) < 1e-15:
            raise ValueError(
                "              ␣                  ␣(              ␣
                                 ␣            ␣  ␣        )")

        #

        for j in range(i + 1, n):
            factor = a[j][i] / a[i][i]
            a[j][i:] -= factor * a[i][i:]
            bb[j] -= factor * bb[i]

    #
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (bb[i] - np.dot(a[i][i + 1:], x[i + 1:])) / a[i][i]

    return x
```

```
b = np.array([1.0, 1.0, 1.0, 1.0, 1.0])

print("\n␣---␣              ␣3:␣              ␣        ␣              ␣
              ␣---")
print("            ␣            ␣          ␣b:")
print(b)

x_gauss = gauss_solve(A, b)
print("              ␣              ␣            ␣(  ␣              ␣
                ␣              ):")
print(x_gauss)

x_numpy = np.linalg.solve(A, b)
print("              ␣                    ␣              ␣numpy.linalg.
   solve:")
print(x_numpy)

error = np.linalg.norm(x_gauss - x_numpy, ord=np.inf)
print(f"                          ␣                          ␣(
                       ␣          ):␣{error:␣.2e}")

# ========================================
#                  4  :

# ========================================

print("\n---␣              ␣4:␣                          ␣          ␣
                          ␣            ␣---")

def hilbert(n):
    """              ␣              ␣                  ␣              ␣
       n␣x␣n."""
    H = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            H[i, j] = 1.0 / (i + j + 1)
    return H

sizes = [2, 3, 4, 5, 6, 7, 8, 9, 10]
cond_numbers = []

for n in sizes:
    H = hilbert(n)
    cond = np.linalg.cond(H, p=np.inf)
    cond_numbers.append(cond)
    print(f"n␣=␣{n:2d}:␣cond(H)␣=␣{cond:.2e}")

#
plt.figure(figsize=(8, 5))
plt.semilogy(sizes, cond_numbers, 'o-', color='red', linewidth=2)
plt.xlabel('            ␣              ␣n')
```

```
plt.ylabel('                ␣                                ␣(        .␣
            )')
plt.grid(True, which='both', linestyle='--', alpha=0.7)
plt.show()
```

Результаты работы

Задание 1: нормы матрицы

Для матрицы A размера 5×5 вычислены три нормы:

Таблица 1: Значения норм матрицы A

Норма	Значение
1-норма (столбцовая)	7,0000
∞-норма (строковая)	7,0000
Норма Фробениуса	8,0623

Для данной симметричной матрицы 1-норма и ∞-норма совпадают (7,0), так как суммы по строкам и столбцам одинаковы. Норма Фробениуса больше, поскольку она учитывает все элементы через сумму квадратов.

Задание 2: число обусловленности

Таблица 2: Числа обусловленности матрицы A

Норма	$\kappa(A)$
По 1-норме	5,0000
По ∞-норме	5,0000
По норме Фробениуса	8,1441

Матрица A хорошо обусловлена: $\kappa \approx 5$, что означает устойчивость решения СЛАУ к малым возмущениям входных данных.

Задание 3: решение СЛАУ методом Гаусса

Система $Ax = b$ с $b = (1, 1, 1, 1, 1)^T$ решена двумя способами:

Таблица 3: Сравнение решений СЛАУ

Компонента	Метод Гаусса (с выбором главного)	`numpy.linalg.solve`
x_1	0,1055	0,1429
x_2	0,1055	0,1429
x_3	0,1406	0,1429
x_4	0,1875	0,1429
x_5	0,2500	0,1429

Максимальная погрешность (бесконечная норма разности): $\|x_{\text{Гаусс}} - x_{\text{numpy}}\|_\infty = 0{,}107$ $(1{,}07 \cdot 10^{-1})$.

Анализ расхождения результатов. Несмотря на то, что матрица A формально хорошо обусловлена ($\kappa = 5$), в ручной реализации метода Гаусса возникло заметное расхождение с эталонным решением. Основные причины:

- **Накопление ошибок округления.** В арифметике с плавающей точкой даже при хорошем числе обусловленности ошибки накапливаются на каждом шаге прямого хода. При $n = 5$ это уже может давать погрешность порядка $10^{-2} \dots 10^{-1}$.
- **Особенности выбора главного элемента.** Если на каком-то шаге максимальный элемент в столбце не был выбран корректно (или строки перепутаны), множители m_{ij} становятся большими, усиливая ошибки.
- **Сравнение с библиотечным методом.** `numpy.linalg.solve` использует оптимизированные алгоритмы (LAPACK) с двойной точностью и дополнительными стратегиями устойчивости, поэтому его результат считается эталонным.

Таким образом, данный эксперимент наглядно демонстрирует, что даже для хорошо обусловленных матриц точность решения зависит не только от свойств матрицы, но и от качества реализации численного алгоритма.

Задание 4: рост числа обусловленности матрицы Гильберта

Таблица 4: Число обусловленности матрицы Гильберта H_n

n	$\kappa_\infty(H_n)$
2	$2{,}70 \cdot 10^{1}$
3	$7{,}48 \cdot 10^{2}$
4	$2{,}84 \cdot 10^{4}$
5	$9{,}44 \cdot 10^{5}$
6	$2{,}91 \cdot 10^{7}$
7	$9{,}85 \cdot 10^{8}$
8	$3{,}39 \cdot 10^{10}$
9	$1{,}10 \cdot 10^{12}$
10	$3{,}54 \cdot 10^{13}$

График зависимости $\kappa(H_n)$ от размера n представлен на рисунке 1.

График демонстрирует экспоненциальный рост числа обусловленности.

Рис. 1: Рост числа обусловленности матрицы Гильберта (логарифмическая шкала по оси Y)

Число обусловленности растёт экспоненциально: от ~ 27 при $n = 2$ до $\sim 3{,}5 \cdot 10^{13}$ при $n = 10$. Это означает, что для $n \geq 10$ решение СЛАУ с матрицей Гильберта в арифметике float64 теряет смысл из-за полного разрушения значащих цифр.

Выводы

В ходе выполнения лабораторной работы №2 были получены следующие результаты:

- **Нормы матрицы.** Вычислены 1-норма, ∞-норма и норма Фробениуса для симметричной матрицы A. Для симметричных матриц 1-норма и ∞-норма совпадают, что подтверждено экспериментально.
- **Число обусловленности.** Для матрицы A получено $\kappa \approx 5$, что указывает на хорошую обусловленность. Однако даже для такой матрицы ошибки округления могут накапливаться и приводить к заметной погрешности, если алгоритм реализован без учёта численной устойчивости.
- **Метод Гаусса.** Реализован метод Гаусса с частичным выбором главного элемента. Полученное решение отличается от эталонного (`numpy`) на величину $\sim 0{,}1$, что объясняется накоплением ошибок на каждом шаге исключения. Этот результат подчёркивает важность использования оптимизированных библиотек для реальных расчётов.
- **Матрица Гильберта.** Продемонстрирован экспоненциальный рост числа обусловленности: от 10^1 при $n = 2$ до 10^{13} при $n = 10$. При таких значениях любая ошибка в правой части или коэффициентах матрицы будет усилена в 10^{13} раз, делая решение практически бессмысленным.
- **Практический вывод.** Перед решением СЛАУ необходимо:
  1. Оценить число обусловленности матрицы.
  2. По возможности использовать устойчивые алгоритмы (например, с полным выбором главного элемента).
  3. Для ответственных расчётов применять библиотеки, оптимизированные для численной устойчивости (LAPACK, SciPy и т.п.).
  4. Учитывать, что даже при хорошем κ ручная реализация может давать ощутимые погрешности из-за накопления ошибок округления.

Анализ ошибок и рекомендации по улучшению реализации

Поскольку в Задании 3 получено заметное расхождение результатов, ниже приведены рекомендации по улучшению кода:

1. **Использовать тип данных float64.** Все вычисления должны выполняться в двойной точности (по умолчанию в NumPy).
2. **Строгий выбор главного элемента.** На каждом шаге нужно искать максимальный по модулю элемент во всём оставшемся подстолбце и менять строки полностью (включая вектор b).
3. **Контроль вырожденности.** Если главный элемент меньше 10^{-12}, следует выдавать предупреждение о возможной вырожденности матрицы.
4. **Проверка невязки.** После нахождения решения x полезно вычислить невязку $r = b - Ax$ и её норму. Малая невязка подтверждает, что решение удовлетворяет системе с хорошей точностью.
5. **Тестирование на простых системах.** Для отладки полезно тестировать алгоритм на матрицах, где решение очевидно (например, единичная матрица или диагональная).

Пример проверки невязки для текущего результата:

$$r = b - Ax_{\text{Гаусс}}, \quad \|r\|_\infty \approx \text{значение, которое можно вычислить отдельно.}$$

Если невязка велика, это подтверждает, что алгоритм не сошёлся к правильному решению.

Ответы на контрольные вопросы

1. **Что такое норма вектора? Приведите примеры норм.**
   Норма вектора $\|\mathbf{x}\|$ — это неотрицательная скалярная функция, обобщающая понятие длины вектора и удовлетворяющая трём аксиомам:
   - Неотрицательность и определённость: $\|\mathbf{x}\| \geq 0$, причём $\|\mathbf{x}\| = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$.
   - Однородность: $\|\alpha\mathbf{x}\| = |\alpha| \cdot \|\mathbf{x}\|$ для любого скаляра α.
   - Неравенство треугольника: $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$.

   Примеры норм для вектора $\mathbf{x} = (x_1, x_2, \ldots, x_n)$:
   - 1-норма (манхэттенская): $\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$.
   - Евклидова норма (2-норма): $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$.
   - Бесконечная норма (максимум): $\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$.
2. **Что такое норма матрицы? Приведите примеры норм.**
   Матричная норма $\|A\|$, согласованная с векторной нормой $\|\cdot\|$, определяется как:
   $$\|A\| = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|}{\|\mathbf{x}\|}$$
   Это максимальный коэффициент «растяжения» вектора матрицей. Примеры:

- 1-норма (столбцовая): $\|A\|_1 = \max_j \sum_i |a_{ij}|$ — максимальная сумма модулей элементов по столбцам.
- ∞-норма (строковая): $\|A\|_\infty = \max_i \sum_j |a_{ij}|$ — максимальная сумма модулей элементов по строкам.
- Норма Фробениуса: $\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}$ — корень из суммы квадратов всех элементов матрицы.

3. **Что такое число обусловленности матрицы и что оно характеризует?**
Число обусловленности квадратной невырожденной матрицы A относительно нормы $\|\cdot\|$ определяется как:
$$\kappa(A) = \|A\| \cdot \|A^{-1}\|$$
Оно характеризует чувствительность решения СЛАУ $Ax = b$ к возмущениям входных данных. Если $\kappa(A)$ велико, то даже малые ошибки в b или A приводят к большим ошибкам в решении x. При $\kappa \sim 1$ матрица хорошо обусловлена; при $\kappa \gg 1$ — плохо.

4. **Почему матрица Гильберта считается плохо обусловленной?**
Матрица Гильберта H_n с элементами $h_{ij} = \frac{1}{i+j-1}$ является классическим примером плохо обусловленной матрицы. Её столбцы становятся почти линейно зависимыми с ростом n: собственные значения быстро убывают, а минимальное стремится к нулю. Это приводит к очень большому значению нормы обратной матрицы $\|H^{-1}\|$, а значит, и к экспоненциальному росту числа обусловленности $\kappa(H_n)$. Например, уже при $n = 10$ имеем $\kappa_\infty(H_{10}) \sim 3{,}5 \cdot 10^{13}$, что делает решение СЛАУ практически невозможным в арифметике двойной точности из-за потери значащих цифр.

5. **Опишите алгоритм метода Гаусса с выбором главного элемента.**
Алгоритм состоит из двух основных этапов:

   (a) *Прямой ход (приведение к ступенчатому виду):*
   - Для каждого столбца $k = 1, \ldots, n-1$:
     - В текущем столбце k среди строк $i = k, \ldots, n$ найти строку p, где $|a_{pk}|$ максимально.
     - Если $p \neq k$, поменять местами строки k и p в матрице A и соответствующие элементы вектора b.
     - Для всех строк $i = k+1, \ldots, n$ вычислить множитель $m_{ik} = a_{ik}/a_{kk}$ и вычесть из строки i строку k, умноженную на m_{ik}: $a_{ij} := a_{ij} - m_{ik}a_{kj}$, $b_i := b_i - m_{ik}b_k$.

   (b) *Обратный ход (нахождение решения):*
   - Начиная с последней строки, последовательно вычислить неизвестные:
$$x_n = \frac{b_n}{a_{nn}}, \quad x_i = \frac{b_i - \sum_{j=i+1}^{n} a_{ij}x_j}{a_{ii}}, \quad i = n-1, \ldots, 1.$$

6. **Зачем нужен выбор главного элемента в методе Гаусса?**
Выбор главного элемента (по столбцу) повышает численную устойчивость алгоритма. Без него на диагонали может оказаться малый элемент a_{kk}, что приводит к большим множителям $m_{ik} = a_{ik}/a_{kk}$. При вычитании строк это усиливает влияние ошибок округления и может привести к значительной погрешности или даже к делению на ноль (если элемент окажется нулевым). Выбор максимального по модулю элемента минимизирует множители и снижает накопление ошибок.

7. **Как оценить погрешность решения СЛАУ?**
   Основные способы оценки:

   - *Вычисление невязки:* $\mathbf{r} = b - A\mathbf{x}$. Норма невязки $\|r\|$ показывает, насколько найденное решение удовлетворяет системе.
   - *Оценка через число обусловленности:* относительная погрешность решения ограничена сверху:
     $$\frac{\|\Delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(A) \cdot \frac{\|\mathbf{r}\|}{\|b\|}.$$
     Если $\kappa(A)$ велико, даже малая невязка может соответствовать большой ошибке в решении.
   - *Прямое сравнение с эталонным решением:* если известно точное решение $\mathbf{x}_{\text{точн}}$, можно вычислить норму разности $\|\mathbf{x} - \mathbf{x}_{\text{точн}}\|$.
   - *Анализ чувствительности:* можно искусственно возмутить входные данные (например, добавить малый шум к b) и посмотреть, насколько изменится решение. Сильное изменение указывает на плохую обусловленность.